

PRD-02

Learning Ecosystem Backend

Version 1.0

Objective

Develop a scalable Learning Management backend that enables students to browse structured roadmaps, learn through topics and lessons, access resources, track their progress, bookmark content, and search efficiently.

The design must support adding new technologies and subjects without requiring code changes.

The system should be entirely data-driven.

Scope

This PRD includes:

- Learning Categories
 - Roadmaps
 - Sections
 - Topics
 - Lessons
 - Learning Resources
 - Progress Tracking
 - Bookmarks
 - Continue Learning
 - Search APIs
-

New Database Models

Category

Examples:

- Programming Languages
- Web Development
- Mobile Development
- AI & ML
- Data Science
- Cyber Security
- Cloud Computing
- DevOps
- Computer Science Core
- Placement Preparation

Fields

- id
 - title
 - slug
 - description
 - icon
 - displayOrder
 - isActive
 - createdAt
 - updatedAt
-

Roadmap

Examples

Python Roadmap

Java Roadmap

React Roadmap

DSA Roadmap

Fields

- id
- categoryId
- title
- slug
- description
- thumbnail
- difficulty
- estimatedHours
- prerequisites

- displayOrder
 - isPublished
 - createdAt
 - updatedAt
-

Roadmap Section

Examples

Python Basics

Functions

OOP

Advanced Python

Fields

- id
 - roadmapId
 - title
 - description
 - order
-

Lesson

Each section contains lessons.

Fields

- id
 - sectionId
 - title
 - slug
 - description
 - estimatedMinutes
 - order
 - isPublished
-

Learning Resource

Each lesson can have multiple resources.

Examples

PDF

Video

Article

GitHub

Documentation

Practice Link

Fields

- id
 - lessonId
 - type
 - title
 - url
 - duration
 - author
 - thumbnail
-

Lesson Progress

Tracks every student's learning.

Fields

- id
 - userId
 - lessonId
 - completed
 - completedAt
 - watchPercentage
 - timeSpent
-

Bookmark

Fields

- id
- userId
- lessonId
- createdAt

Recently Viewed

Fields

- id
 - userId
 - lessonId
 - viewedAt
-

Relationships

Category

|



Roadmap

|



Section

|



Lesson

|



Resources

User

|

└─ Lesson Progress

└─ Bookmarks

CRUD APIs

Categories

GET /api/categories

GET /api/categories/:id

POST /api/categories

PUT /api/categories/:id

DELETE /api/categories/:id

(Admin protected for create/update/delete)

Roadmaps

GET /api/roadmaps

GET /api/roadmaps/:id

POST /api/roadmaps

PUT /api/roadmaps/:id

DELETE /api/roadmaps/:id

Support filters:

- Category
- Difficulty
- Search
- Published

Pagination is mandatory.

Sections

GET /api/sections/:roadmapId

POST /api/sections

PUT /api/sections/:id

DELETE /api/sections/:id

Lessons

GET /api/lessons/:sectionId

GET /api/lesson/:id

POST /api/lesson

PUT /api/lesson/:id

DELETE /api/lesson/:id

Resources

GET /api/resources/:lessonId

POST /api/resources

PUT /api/resources/:id

DELETE /api/resources/:id

Student APIs

Continue Learning

GET /api/learning/continue

Returns the most recently viewed incomplete lesson.

Mark Lesson Complete

POST /api/lesson/:id/complete

Updates:

- completed
 - completion time
 - percentage
-

Update Progress

PATCH /api/lesson/:id/progress

Body

```
{  
  
  "watchPercentage": 65,  
  
  "timeSpent": 420  
}
```

Bookmarks

POST /api/bookmark/:lessonId

DELETE /api/bookmark/:lessonId

GET /api/bookmarks

Recently Viewed

Automatically update whenever a lesson is opened.

GET /api/recent

Search

Global search endpoint:

GET /api/search?q=

Should search across:

- Categories
- Roadmaps
- Lessons

Return grouped results.

Business Rules

- Only published content is visible to students.
 - Admins can access drafts.
 - Display order determines the sequence.
 - Deleting a roadmap should cascade to sections, lessons, and resources.
 - Users can only update their own progress.
 - Duplicate bookmarks are not allowed.
 - One progress record per user per lesson.
-

Validation

Use Zod.

Validate:

- Slugs must be unique.
 - Titles cannot be empty.
 - URLs must be valid.
 - Estimated time must be positive.
 - Display order cannot be negative.
-

Authorization

Student:

- View content
- Track progress
- Bookmark
- Search

Mentor:

- Student permissions
- Can review progress (future extension)

Admin:

- Full CRUD
 - Publish/unpublish content
 - Reorder content
-

Performance Requirements

- Implement pagination for all list endpoints.
 - Support filtering and sorting.
 - Add indexes on:
 - slug
 - categoryId
 - roadmapId
 - sectionId
 - lessonId
 - userId
 - Avoid N+1 queries.
 - Use Prisma's relation loading efficiently.
-

API Response Standards

Continue using the PRD-01 response format:

```
{  
  
  "success": true,  
  
  "message": "Roadmaps fetched successfully",  
  
  "data": {},  
  
  "errors": null  
}
```

Acceptance Criteria

PRD-02 is complete when:

- All new Prisma models are implemented with proper relationships.
- Database migrations run successfully.
- CRUD APIs for categories, roadmaps, sections, lessons, and resources are functional.
- Progress tracking, bookmarks, recently viewed, and continue learning features work correctly.
- Global search returns grouped results.
- Role-based authorization is enforced.
- Validation, pagination, filtering, and sorting are implemented.
- API documentation is updated in Swagger.
- Logging and error handling follow the standards established in PRD-01.

- All new endpoints compile without TypeScript errors and integrate cleanly with the existing architecture.
-

One architectural recommendation

Before you start PRD-02, make one small change that will save a lot of work later.

Instead of creating generic **Lessons**, introduce a `contentType` field (or similar) in the Lesson model, for example:

- `NOTE`
- `VIDEO`
- `ARTICLE`
- `QUIZ`
- `ASSIGNMENT`
- `PROJECT`
- `CODING_PROBLEM`

This lets you reuse the same learning structure across the entire platform. A Python roadmap, a DBMS roadmap, an interview preparation roadmap, or even a project roadmap can all use the same hierarchy without redesigning the database later. It's a small design decision now that gives you much more flexibility as the platform grows.